Chapter 9
Dictionaries

Computer Science with Python — Class XI

In This Chapter

1. Introduction
2. Dictionary — Key:Value Pairs
3. Working with Dictionaries
4. Dictionary Functions and Methods

9.1 Introduction

Python offers many ways to organize *collections*:

- strings, lists, tuples (already covered)
- **dictionaries** (this chapter)

You will learn:

- how items are organized
- how to access items
- operations on dictionaries
- related functions and methods

9.2 Dictionary — Key:Value Pairs

A dictionary is a collection of **key:value pairs**.

- Lists/strings use an *index*
- Dictionaries use a *key* to find a *value*

Analogy: an English dictionary

- word $\leftrightarrow$ key
- meaning $\leftrightarrow$ value

Definition

Dictionaries are mutable, unordered collections of key:value pairs that associate keys to values. Also called associative arrays, mappings, or hashes.

Creating a Dictionary

```
<name> = { <key>:<value>, <key>:<value>, ... }
```

```
teachers = {
    "Dimple"   : "Computer Science",
    "Karen"    : "Sociology",
    "Harpreet" : "Mathematics",
    "Sabah"    : "Legal Studies"
}
```

- curly braces mark start and end
- colon separates key and value
- commas separate pairs

Key–Value Table

Pair	Key	Value
Dimple : CS	Dimple	Computer Science
Karen : Sociology	Karen	Sociology
Harpreet : Maths	Harpreet	Mathematics
Sabah : Legal Studies	Sabah	Legal Studies

NOTE

Internally, dictionaries are indexed on the basis of keys.

More Examples

```
dict1 = {}    # empty dictionary

DaysInMonths = {
    "January": 31, "February": 28, "March": 31,
    "April": 30,   "May": 31,      "June": 30
}

BirdCount = {
    "Finch": 10, "Myna": 13, "Parakeet": 16,
    "Hornbill": 15, "Peacock": 15
}
```

Keys Must Be Immutable

Allowed keys: string, number, tuple of immutable items.

```
>>> dict3 = {[2,3]:"abc"}
TypeError: unhashable type: 'list'
```

IMPORTANT

`TypeError: unhashable type` means you used a *mutable* object as a key. Dictionaries do not allow that.

Accessing Elements

Syntax: `dictionary[key]`

```
>>> teachers["Karen"]
'Sociology'
>>> print("Karen teaches", teachers["Karen"])
Karen teaches Sociology
```

Printing the name alone shows the whole dictionary (order is *not* guaranteed).

Lookup

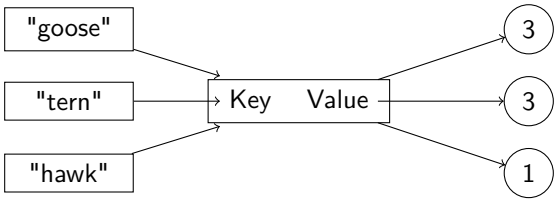
An operation that takes a key and finds the corresponding value.

Missing Keys Raise KeyError

```
>>> d = {"Vowel1":"a", "Vowel2":"e"}
>>> d[13]
KeyError: 13
```

Ensure the key exists before accessing it with `[]`.

References of Keys and Values



extitFig. 9.1 — keys and values stored via references

Traversing a Dictionary

A `for` loop assigns *keys* one by one:

```
d1 = {5: "number", "a": "string", (1,2): "tuple"}

for key in d1:
    print(key, ":", d1[key])
```

Possible output (order may vary):

```
a : string
(1, 2) : tuple
5 : number
```

Program 9.1 — Phone Dictionary

```
PhoneDict = {
    "Madhav": 1234567, "Steven": 7654321,
    "Dilpreet": 6734521, "Rabiya": 4563217
}
for name in PhoneDict:
    print(name, ":", PhoneDict[name])
```

Printed order need not match insertion order (dictionaries are unordered collections in this chapter's model).

keys() and values()

```
>>> d.keys()
dict_keys(['Vowel5', 'Vowel4', ...])
>>> d.values()
dict_values(['u', 'o', 'i', 'e', 'a'])

>>> list(d.keys())
['Vowel5', 'Vowel4', ...]
>>> list(d.values())
['u', 'o', 'i', 'e', 'a']
```

Characteristics of a Dictionary

1. **Unordered set** of key:value pairs
2. **Not a sequence** — no positional index range
3. **Indexed by keys**, not numbers
4. **Keys must be unique**
5. **Mutable** — values can change in place
6. Internally stored as **mappings** (hash function)

Mixed Immutable Keys

```
dict1 = {
    0: "Value for key 0",
    "3": "Value for a string-as-a-key",
    (4, 5): "Value for a tuple-as-a-key"
}
>>> dict1[0]
>>> dict1[(4, 5)]
>>> dict1[4, 5]      # same tuple key
```

Values may be *any* object; keys must be immutable.

Unique Keys, Shared Values

```
BirdCount = {
    "Finch": 10, "Hornbill": 15, "Peacock": 15
}
```

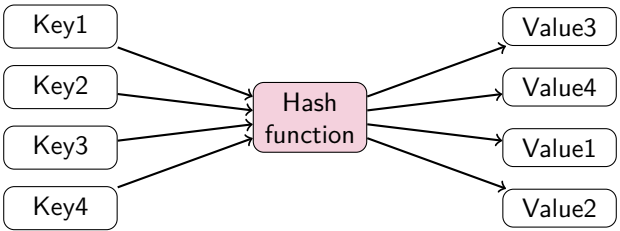
`Hornbill` and `Peacock` are different keys with the *same* value 15.

Mutation and Adding Pairs

```
>>> dict1["3"] = "Changed to new string"
>>> dict1["new"] = "a new pair is added"
```

- existing key $\rightarrow$ value is **updated**
- new unique key $\rightarrow$ pair is **added**

Internal Mapping



extitFig. 9.2 — keys mapped to values

Ways to Create Dictionaries

1. Literal

```
Employee = {'name':'John', 'salary':10000, 'age':24}
```

2. Empty + assignment

```
Employee = {}           # or dict()
Employee['name'] = 'John'
Employee['salary'] = 10000
```

dict() Constructor

```
dict(name='John', salary=10000, age=24)
dict({'name':'John', 'salary':10000, 'age':24})
dict(zip(('name','salary','age'), ('John',10000,24)))
dict([['name','John'], ['salary',10000], ['age',24]])
```

NOTE

`dict()` is slower than `{...}`. Prefer literals unless you need the constructor.

Adding vs Updating

Same syntax:

```
Employee['dept'] = 'Sales'      # add if new
Employee['salary'] = 20000      # update if exists
```

Remember

Add: key must *not* exist. Update: key *must* exist (otherwise a new pair is created).

Nesting Dictionaries

A dictionary may be stored as a *value*, never as a *key*.

```
Employees = {
    'John': {'age': 25, 'salary': 20000},
    'Diya': {'age': 35, 'salary': 50000}
}
for key in Employees:
    print(Employees[key]['age'])
    print(Employees[key]['salary'])
```

Program 9.3 — Interactive Creation

```
n = int(input("How many students ? "))
CompWinners = {}
for i in range(n):
    key = input("Name of the student : ")
    value = int(input("Number of wins : "))
    CompWinners[key] = value
print(CompWinners)
```

Deleting Elements

del

```
del emp13['age']      # KeyError if missing
```

pop() — deletes and *returns* the value

```
>>> employee.pop('age')
24
>>> employee.pop('new', "Not Found")
'Not Found'
```

Membership: in / not in

They test **keys only**.

```
>>> 'age' in emp1           # True
>>> 'John' in emp1          # False  (it's a value)
>>> 'xyz' in dict1.values()  # search values
```

Pretty Printing

```
import json
print(json.dumps(Winners, indent=2))
```

```
{
  "Nihar": 3,
  "Rohan": 5,
  "Zeba": 3
}
```

Frequency Count + split()

```
sentence = "This is a super idea This idea ..."
words = sentence.split()
d = {}
for one in words:
    if one not in d:
        d[one] = words.count(one)
```

`split()` breaks text on whitespace (or a given separator).

len() and clear()

```
len(employee)           # number of pairs

employee.clear()        # empty dict remains
del employee            # object gone; NameError after
```

clear vs del

`clear()` → empty dictionary object still exists.
`del D` → name is undefined.

get()

```
emp11.get('dept')
'Sales'
emp11.get('desig', "Not yet")
'Not yet'
```

Safer than `D[key]` when the key may be missing.

items(), keys(), values()

```
for ky, vl in employee.items():
    print(vl, ky)

employee.keys()
employee.values()
```

Returned sequences have no guaranteed order (as taught here).

update()

```
employee1.update(employee2)
```

Merges pairs from `employee2` into `employee1`. Same keys are **overwritten**. Equivalent to:

```
for k in newDict.keys():
    dictionary[k] = newDict[k]
```

Let Us Revise

- Mutable key:value collection; also called mapping / hash
- Keys: immutable and unique
- Unordered — not a sequence (no slice / concat)
- Create with `{}` or `dict()`
- Add / update share syntax; delete with `del` or `pop()`
- Methods: `len`, `clear`, `get`, `items`, `keys`, `values`, `update`
- `in` / `not in` test keys only

Glossary

Dictionary	Mutable unordered collection of key:value pairs
Mapping	Linking a key to a value via a hash function
Nesting	A dictionary stored as a value inside another
Lookup	Finding the value for a given key

Solved Ideas (quick)

- Lists vs dicts: ordered sequence vs named keys
- Lists cannot be keys (mutable / unhashable)
- Dicts grow freely — no “size overflow” error
- Reverse lookup: loop values (or use `.values()`)
- Nested dicts for student / employee records

Check Point 9.1

1. Why are dictionaries mutable types?
2. Ways of creating dictionaries?
3. What can be stored as values? As keys?
4. How is indexing different from a list?
5. Which types may be keys? (str, tuple, int, float, list, dict)
6. How do you add / remove pairs?

Assignments Snapshot

Type A — conceptual (unordered, keys, `unhashable`, `clear` vs `del`)

Type B — errors and outputs (`D`+3, list-as-key, nested dict as key, `in` on values)

Type C — programs: teams W–L, product prices, months/days, invert a mapping, overlapping keys

End of Chapter 9

Thank you